

Chapter 7. The Data Landscape

The four questions every data job answers, the ten roles that answer them, how teams are organized, and one request followed through every role.

Draft chapter

16 September 2026

Every query and number checked

Contents

Chapter 7. The Data Landscape

Why this matters

In plain English

7.1 The four questions every data job answers

7.2 The tracks and the ten roles

7.3 The shape of the field: a trunk that branches

7.4 The automation and integration track

7.5 How data teams are organized

7.6 One request, every role

7.7 How AI assistants are changing each role

7.8 Where you are right now

Common mistakes and how to spot them

In the real world: Anita wants to hire a data scientist

Tools

The project: map the data work around you

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 7. The Data Landscape

Part I — The Map

Chapter at a glance

You will learn to: name the four questions every data job answers, and place any job title under one of them · describe what each of the ten main data roles does, what it produces, and which tools it uses · explain why the field grows like a tree, with a shared trunk and branches that rejoin at the top · recognize the automation and integration track, and the automation hidden inside every other role · compare centralized, embedded, and hub-and-spoke data teams, and say when each fits · follow one business request through every role, from the first question to a running platform · judge what AI assistants change in each role, and what stays your responsibility.

Before you start: Chapter 1 (what data is) and Chapter 2 (files, databases, servers, and APIs). Chapter 3 (how a business runs on data) helps but isn't required.

Time needed: 2–3 hours, including the exercises and the project.

Tools: a pen and paper, or any free drawing tool for the project. No software to install.

Practice data: one short query on the Riverstone one-year database (`riverstone_2025`), shown so you can see what an analyst's answer looks like. You don't need to run it yet; Chapter 12 teaches every line. Every result and number shown is real.

Why this matters

From the outside, "working in data" sounds like a country where every city has the same name. Data analyst, business analyst, BI developer, analytics engineer, data scientist, ML engineer, data engineer, AI engineer, integration engineer, data architect: the titles blur together, and the job descriptions overlap. People new to the field often spend months learning tools for a job they turn out not to want, or apply for roles whose real work they don't understand.

The first skill in a data career isn't SQL or Python. It's **orientation**: knowing what the roles are, what each one produces, how they depend on each other, and where you stand among them. Once you have that, every later decision gets easier. You can read a job description and see the real job behind the title. You can pick what to learn next for a reason. And in an interview, you can explain where your work fits in the team, which interviewers notice.

This chapter is the map. Chapter 8 turns it into a route, skill by skill, and Chapter 9 is honest about how long the route takes.

In plain English

Think about what happens when you walk into a hospital with a pain in your side.

- At reception, someone asks what's wrong and writes it down in a way the doctors can use. Vague ("it hurts") becomes precise ("sharp pain, lower right, since yesterday"). That's the work of a **business analyst**: turning a vague request into a clear question.
- A doctor looks at your test results and says what's happening now. That's a **data analyst**: describing what happened, clearly and correctly.
- A specialist uses your history to judge what will probably happen next, and which treatment will work. That's a **data scientist**: prediction and cause.
- The lab technicians make sure every blood sample is collected, labeled, tested, and delivered to the right doctor on time. Nobody thanks them until a sample goes missing. That's a **data engineer**.
- The hospital's systems send your prescription to the pharmacy and a reminder to your phone, without anyone re-typing it. That's **automation and integration**.
- Someone designed the building: where the lab sits relative to the emergency ward, how records flow between departments, who is allowed to see what. That's a **data architect**.

One patient, one question, many specialists, all depending on each other. A data team works the same way. Keep this picture in mind; section 7.6 follows a single Riverstone request through every one of these roles.

7.1 The four questions every data job answers

Strip away the tools and the titles, and every data role exists to help answer one of four questions about an organization and its data.

1. **What happened?** Describing the past and present clearly: last month's sales, this week's late deliveries, which customers have stopped ordering, and the immediate reasons behind them.
2. **What will happen, and why?** Predicting what comes next, and finding out what causes what: which customers are likely to leave, how much stock to make for Diwali, whether a new discount really increased sales.
3. **How does data move, get served, and get put to work, reliably?** Building the machinery: collecting data from every system, checking it, storing it, and delivering it (as a report, an alert, or a task in another system) on time, every time, without someone doing it by hand.
4. **How should the whole system be designed?** Deciding how all the pieces fit together, who owns what, what it costs, and how it stays secure and trustworthy as the company grows.

Almost every data job is a specialization of one of these. When you meet an unfamiliar title, ask *"Which of the four questions does this person spend most of their day on?"* The title then stops being confusing.

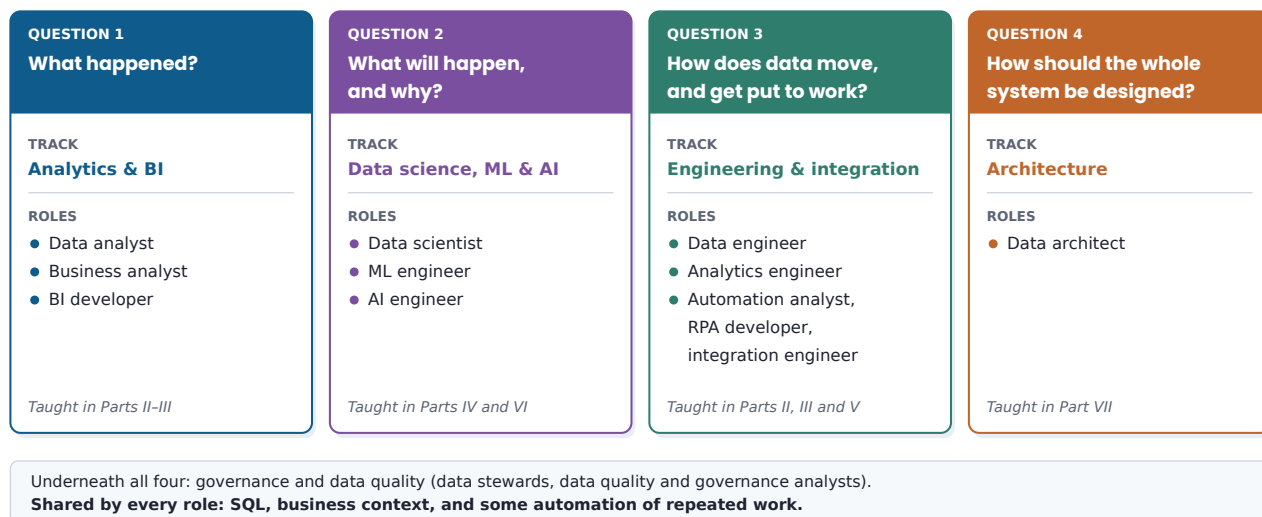


Figure 7.1 – The four questions, the tracks that answer them, and the ten roles this book follows. Notice the band at the bottom: some skills belong to everyone.

Simplification note. Real jobs cross these lines. Analysts ask "why?" all the time, and a good analyst explains *why* sales fell in March, not only *that* they fell. The difference is depth: an analyst finds the likely reason in the data; a data scientist designs an experiment or a model to prove a cause or predict an outcome. Use the four questions to find a role's center of gravity, not its edges.

7.2 The tracks and the ten roles

A **track** is a family of related roles that answer the same question. A **role** is a job with a recognizable set of responsibilities, whatever a particular company calls it. The same role might be advertised as "MIS executive", "reporting analyst", or "insights analyst", and the same title might mean different work at two companies. That's why this book describes roles by what they produce, not by title.

Here are the ten roles this book follows, grouped by track in the same order as Figure 7.1.

Track	Role	Center of gravity	What they typically produce	Core tools	Taught in
Analytics & BI	Data analyst	What happened, and why it probably happened	Answers to business questions, reports, analyses, recommendations	Excel or Google Sheets, SQL, a BI tool, some Python	Part II
Analytics & BI	Business analyst	What the business needs, and how a process should change	Clear requirements, process maps, user stories, test plans, the case for a change	Spreadsheets, SQL, process-mapping and ticketing tools	Ch 24–25

Track	Role	Center of gravity	What they typically produce	Core tools	Taught in
Analytics & BI	BI developer	Making "what happened" available to everyone, all the time	Dashboards, data models behind them, scheduled reports	Power BI or similar, SQL, DAX	Ch 16, 28
Data science, ML & AI	Data scientist	What will happen, and what causes what	Models, forecasts, experiment results, recommendations	Python, statistics, scikit-learn, SQL	Part IV
Data science, ML & AI	ML engineer	Keeping models working in the real world	Models running as services, with monitoring and retraining	Python, cloud services, MLOps tools	Ch 56
Data science, ML & AI	AI engineer	Building useful, safe products on top of AI models	Assistants, AI features, and AI steps inside business workflows, with evaluation	Python, LLM APIs, retrieval, evaluation tools	Ch 54–58
Engineering & integration	Data engineer	Getting data from every source, correct and on time	Pipelines, warehouses, data-quality checks	SQL, Python, orchestration tools, cloud	Part V
Engineering & integration	Analytics engineer	One trusted, tested version of the business's numbers	Clean, documented, tested tables that analysts and dashboards use	SQL, dbt, Git	Ch 32
Engineering & integration	Automation analyst, RPA developer, integration engineer	Removing manual steps, and connecting systems so data flows between them	Automated reports, low-code flows, bots, API integrations, data written back into business systems	Spreadsheet automation, Python, Power Automate or n8n, APIs	Ch 19–20, 51, 58
Architecture	Data architect	How the whole data system should be designed	Designs, standards, decision records, platform roadmaps	All of the above, at the level of design	Part VII

A few terms in that table will be new. A **dashboard** is a screen of charts and numbers that refreshes itself from the data. A **model**, in data science, is a set of rules learned from past data that makes predictions about new data. A **pipeline** is an automated series of steps that moves data from one place to another and changes it along the way. **BI** stands for business intelligence: the tools and practices for turning data into reports and dashboards people use to run the business. **MLOps** and **LLM** will each get full chapters; for now, MLOps is "keeping machine learning models healthy in production", and an LLM (large language model) is the kind of AI model behind chat assistants.

The one skill every role shares

Look down the *Core tools* column and one name keeps appearing: **SQL**. It's the language for asking questions of a database, and every role on this map writes it, often every day. The analyst uses it to answer questions. The BI developer uses it to feed dashboards. The analytics engineer builds whole

tested models of the business in it. The data scientist pulls training data with it. The data engineer moves and checks data with it. The integration engineer uses it to choose which records to push into the CRM. The architect reads it to understand what the system is really doing. That's why Part II teaches it at full depth in Chapters 12 and 13, and why nothing you learn there goes to waste, whichever branch you climb.

Watch out: titles lie, outputs don't. A "data scientist" at one company builds dashboards all day; a "business analyst" at another writes Python pipelines. Before you decide a job is (or isn't) for you, read what the person will *produce* each week. Chapter 8 shows how to decode a job description line by line.

Roles beyond the ten

The map has more cities than this book can visit in depth. **Data governance analysts** and **data stewards** make sure data is defined, documented, protected, and used lawfully. **Data quality analysts** hunt for and fix errors at the source. Specialist scientists work on text (NLP), images (computer vision), or recommendations. **Analytics managers**, **heads of data**, and **chief data officers** lead the people. Many domain roles combine a business function with data work: sales operations, finance analyst, supply chain analyst, marketing analyst. Each of these sits on the same map, and the chapters listed in "Where this leads" show where the book touches them.

7.3 The shape of the field: a trunk that branches

Here is the most important idea in this chapter: **the roles share a trunk and branch upward, and the branches rejoin at the top.**

Everyone starts with the same foundations: understanding data (Part 0), a spreadsheet, SQL, and business sense. The analyst core grows straight up from there (Part II), and many people build a whole career on it. Then the field branches. After the advanced analytics and analytics engineering skills of Part III, one branch leads toward data science and machine learning (Part IV), and the other toward data engineering and integration (Part V). The two branches meet again when models and AI have to run inside a real business (Part VI), and they come together fully at the top, in architecture (Part VII). A data architect can't design a system without having walked at least partway down each branch.

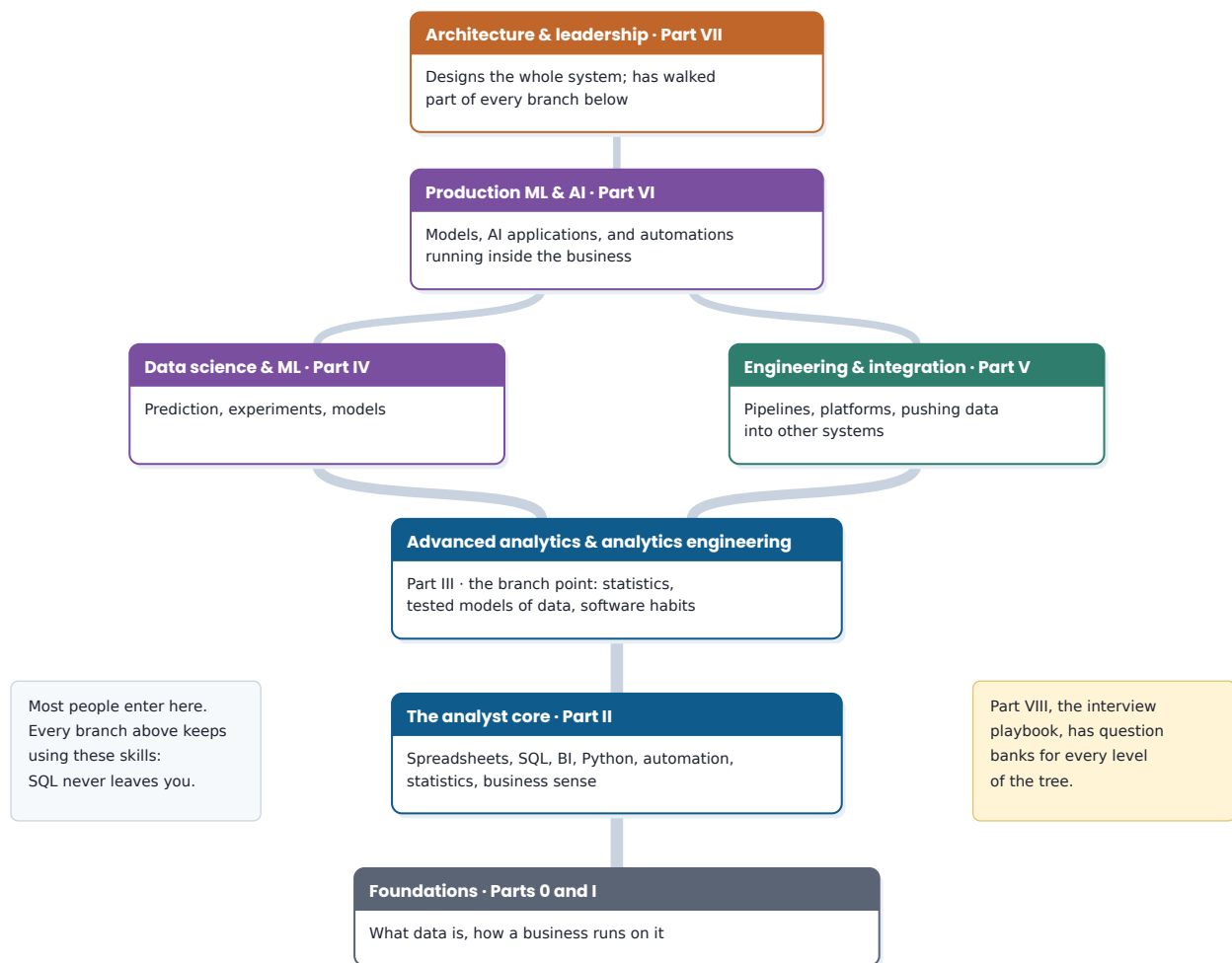


Figure 7.2 – The field grows like a tree, and this book follows it part by part. Read it from the bottom up: nobody starts at the top.

Two consequences follow, and they shape how you should use this book.

- **You don't abandon lower levels when you climb.** A data scientist still writes SQL every week, and an architect still reads spreadsheets. Skills accumulate.
- **Science and engineering are peers, not a ladder.** Neither branch is "above" the other. The book teaches data science (Part IV) before data engineering (Part V) because many analysts move toward science first, but plenty of excellent careers go straight from the analyst core to engineering. Chapter 8 turns this tree into tiers you can plan with, in the same order as the book's parts.

7.4 The automation and integration track

At every level of the tree, the same practical question comes up: *once the data is right, how does it reach the people and systems that act on it, without someone doing it by hand?*

In most companies, a large share of data work is exactly this. Someone exports a file from one system, pastes it into a spreadsheet, fixes the columns, adds formulas, saves a copy, and emails it to twenty managers. Next week they do it again. Chapter 2's "Friday file" was a small example of that kind of routine. Removing those steps is the job of the automation and integration track.

The track has three common roles, and the lines between them are blurry.

- An **automation analyst** finds manual, repeated work in business processes and automates it, often with spreadsheet automation (macros, VBA, Office Scripts, Google Apps Script), low-code tools such as Power Automate or n8n, and Python scripts. The role often grows out of an analyst or operations job.
- An **RPA developer** builds software "robots" that click through screens the way a person would. **RPA** stands for robotic process automation. It's the right tool when an old system has no other way in, and the wrong tool when a proper connection exists, because a robot breaks whenever the screen changes.
- An **integration engineer** connects systems directly, usually through APIs (Chapter 2), so data flows between them: new orders from the website into the ERP, payment status from finance into the CRM, a risk flag from the data warehouse back into the sales team's tools. Writing data from the warehouse back into business systems is called **reverse ETL** or **data activation**, and Chapter 51 teaches it.

Here's what makes this track different from the others: **every data role includes some automation**. The analyst schedules a refreshable report instead of rebuilding it. The BI developer sets up a dashboard subscription. The data engineer's whole job is automated pipelines. The ML engineer automates retraining. The architect decides which automations the company should build, own, and eventually retire. The automation roles specialize in it, but nobody in data is exempt.

Take a simple example. Riverstone's **Daily Sales Flash**, a one-page summary of yesterday's orders, could start as a manual report that takes someone 40 minutes each working day. If it's produced on 250 working days a year, that's $40 \times 250 = 10,000$ minutes, or about 167 hours: more than four working weeks spent re-typing the same report. Over the course of this book, that report becomes a SQL query (Chapter 12), a Python script (Chapter 18), a VBA macro and an Apps Script project (Chapter 19), a scheduled email in every manager's inbox (Chapter 20), a monitored pipeline (Chapters 46–47), a source of flags pushed into the CRM (Chapter 51), and finally part of a designed reporting platform (Chapter 63). The business need never changes. The way it's met gets better at each level, which is exactly how the job grows in real life.

AUTOMATED FLOW

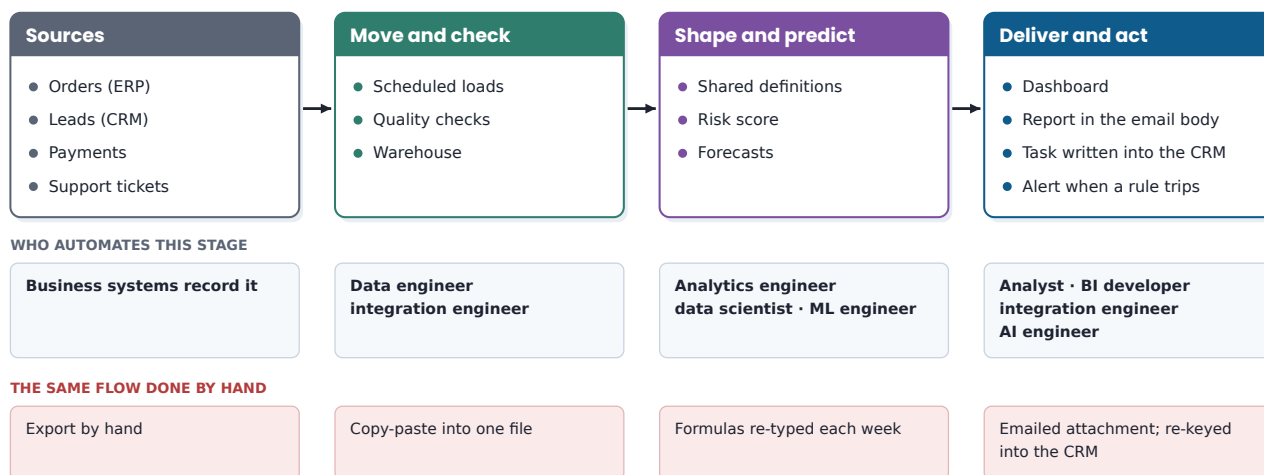


Figure 7.3 — The flow from source to action, with the roles that automate each stage. The red row is what the same flow looks like when people do it by hand.

Real-life example: the report nobody owns. Many companies run an important report on a macro written years ago by someone who has since left. It works until the source file's columns change, and then it fails quietly or, worse, produces wrong numbers that look right. Automating a report is only half the job. Someone has to own it, monitor it, and know what to do when it breaks. Chapter 63 covers that governance at the architect level.

7.5 How data teams are organized

Knowing the roles is half the map. The other half is how companies arrange those people, because it changes your day, your boss, and your career path. There are three common structures.

In a **centralized** team, all the data people sit in one team, usually under a head of data or a technology leader, and serve every department. Sales, finance, and operations send requests to that team.

In an **embedded** (or **decentralized**) setup, data people sit inside the departments they serve and report to those departments' heads: a sales analyst in the sales team, a finance analyst in finance. There's little or no central data team.

In a **hub-and-spoke** (sometimes called **federated**) structure, a central hub owns the shared platform, the standards, and the common definitions of key numbers, while analysts in each department (the spokes) do the day-to-day work. Spokes often report to their department but follow the hub's standards and meet with the hub regularly.

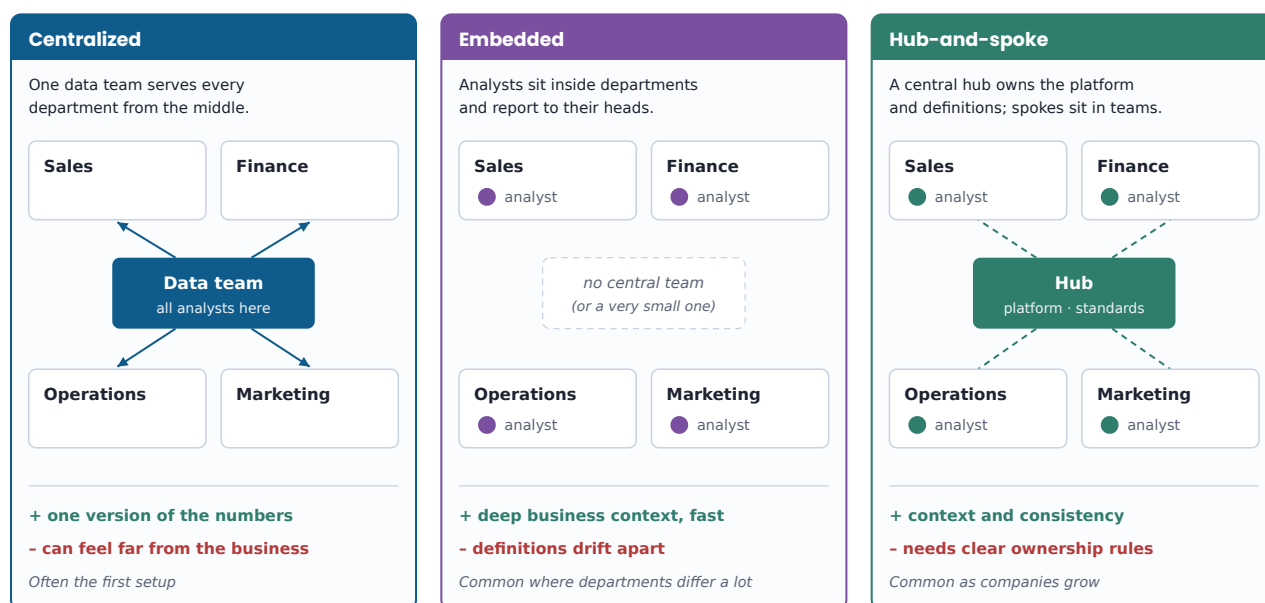


Figure 7.4 — Three ways to organize a data team. The same people can be arranged very differently, and each arrangement has a typical failure.

	Centralized	Embedded	Hub-and-spoke
Consistency of numbers	High: one team, one definition	Low: each department defines "revenue" its own way	High for shared numbers, if the hub enforces definitions
Understanding of the business	Can be weak; the team hears about problems secondhand	Strong; analysts sit in the meetings	Strong in the spokes
Speed for a department	Slower; requests wait in a queue	Fast	Fast for local work; shared changes take coordination
Duplicated work	Low	High: three teams build three versions of the same report	Medium
Career growth for data people	Clear; you learn from other data people	Harder; your manager may not understand your work	Clearer, through the hub's community and standards
Typical failure	The bottleneck: "the data team never gets to our request"	"Why do sales and finance report different revenue?"	Nobody is sure who owns a definition

No structure is right for every company. Small companies often start with one or two data people who behave like a tiny central team. As departments grow, analysts get hired into them, and inconsistent numbers start to hurt. Many companies then move toward hub-and-spoke to keep local speed while fixing definitions centrally. Chapter 66 goes deeper into building and structuring data teams.

Try it. Think of a company you know, or your college's administration. Where do the people who make reports sit? Which of the three structures is it closest to, and which typical failure have you heard people complain about?

7.6 One request, every role

Roles make the most sense in motion. So let's follow a single business request through all ten of them.

Simplification note. Riverstone is a mid-sized company, and in real life one or two people would cover most of these roles. To see each role clearly, imagine a larger version of Riverstone with one specialist in every seat. The data in step 2 is the real one-year database; the later steps describe what each role would build, and the chapters named teach you to build it.

It's the year-end review, and Anita Rao, the Sales Head, says:

"Some customers seem to have stopped ordering. Which ones, and what should we do about it?"

Step 1: The business analyst turns the request into a question

"Stopped ordering" can mean many things. The business analyst asks questions before anyone touches data: *Stopped since when? Does a cancelled order count? What about customers who signed up and never ordered at all? Who will act on the list, and what will they do? How often do you need it?*

The answers become a short requirement: **"List every customer with no non-cancelled order in the 60 days up to 31 December 2025, including customers who have never ordered. Sales reps will call them. Needed weekly."** That sentence took twenty minutes of conversation, and it will save days of rework.

Step 2: The data analyst answers it

The data analyst writes a query against the one-year database. You don't need to read SQL yet; look at the shape of the question in the code, then at the answer.

```
SELECT c.customer_name,
       c.segment,
       MAX(o.order_date)           AS last_order_date,
       DATE '2025-12-31' - MAX(o.order_date) AS days_since_last_order
FROM customers AS c
LEFT JOIN orders AS o
  ON c.customer_id = o.customer_id
 AND o.status <> 'Cancelled'
GROUP BY c.customer_id, c.customer_name, c.segment
HAVING MAX(o.order_date) < DATE '2025-11-01'
      OR MAX(o.order_date) IS NULL
ORDER BY last_order_date NULLS FIRST;
```

customer_name	segment	last_order_date	days_since_last_order
Home Plus	Retail		
City Needs Store	Retail	2025-03-22	284
Sunrise Caterers	Hospitality	2025-06-10	204
Om Sai Provisions	Retail	2025-07-22	162
Tasty Tiffins	Hospitality	2025-10-26	66

(5 rows)

How it works, in plain words.

- The query starts from every customer, then looks up each one's orders, ignoring cancelled ones.
- For each customer, it finds the latest order date, and counts the days from that date to 31 December 2025.
- It keeps only customers whose latest order was before 1 November, plus customers with no orders at all.
- The blank dates are Home Plus: it signed up on 18 June 2025 and has never ordered, so there's no date to show.

Check one row by hand. Tasty Tiffins last ordered on 26 October 2025. From 26 to 31 October is 5 days, November has 30, and December has 31: $5 + 30 + 31 = 66$ days. ✓

A good analyst doesn't stop at the list. Looking closer, City Needs Store and Tasty Tiffins have placed only two orders each, so it's hard to call them "regulars". Sunrise Caterers and Om Sai Provisions placed four orders each, roughly every five to six weeks, and then went silent. Together those two brought in ₹114,072.50 of revenue in 2025, about 2.6% of Riverstone's ₹4,335,471 for the year.

What to tell Anita. Five customers haven't ordered in 60 days. Two of them, Sunrise Caterers and Om Sai Provisions, used to order regularly, so they're the most urgent calls: find out what went wrong. Home Plus signed up in June and never ordered: a lead that was never converted. The other two ordered only twice, so they may be occasional buyers.

SQL link. Chapter 12 teaches every clause in this query and includes a version of it on the mini database (section 12.15). Chapter 13 improves the rule itself, comparing each customer's silence with their own usual ordering rhythm (Pattern 6).

Dialect note. The same question in MySQL needs two changes: `DATEDIFF` instead of subtracting dates, and a manual sort to put the blank dates first, because MySQL has no `NULLS FIRST`. The result is the same five customers.

```
SELECT c.customer_name,
       c.segment,
       MAX(o.order_date) AS last_order_date,
       DATEDIFF('2025-12-31', MAX(o.order_date)) AS days_since_last_order
FROM customers AS c
LEFT JOIN orders AS o
      ON c.customer_id = o.customer_id
      AND o.status <> 'Cancelled'
GROUP BY c.customer_id, c.customer_name, c.segment
HAVING MAX(o.order_date) < '2025-11-01'
      OR MAX(o.order_date) IS NULL
ORDER BY last_order_date IS NOT NULL, last_order_date;
```

customer_name	segment	last_order_date	days_since_last_order
Home Plus	Retail	NULL	NULL
City Needs Store	Retail	2025-03-22	284
Sunrise Caterers	Hospitality	2025-06-10	204
Om Sai Provisions	Retail	2025-07-22	162
Tasty Tiffins	Hospitality	2025-10-26	66

Step 3: The BI developer makes it available every day

Anita likes the answer and wants it every week without asking. The BI developer adds an "At-risk customers" page to the sales dashboard. It refreshes from the database each morning, each rep can filter it to their own customers, and Anita gets it by email every Monday through a dashboard subscription. The analyst no longer has to re-run the query by hand.

Step 4: The analytics engineer makes the definition trustworthy

A month later, finance mentions that *their* "inactive customers" report uses 90 days, not 60. The customer-support team uses a third rule. Three reports, three answers, and every meeting starts with an argument about whose number is right.

The analytics engineer agrees one definition with sales and finance, writes it once as a tested, documented table (a `customer_activity` model, in the style of Chapter 32), and points every report at it. Tests run automatically: no customer can appear twice, every customer must have a status, and the total number of customers must match the source system. "Active customer" now means one thing everywhere.

Step 5: The data scientist asks what will happen next

The list tells Riverstone who *has* gone quiet. The data scientist asks a better question: *can we tell who is **about to** go quiet, while there's still time to help?* They look for early warning signs in the history: orders getting smaller, gaps getting longer, late payments, complaints to support. They build a model that gives each customer a risk score, and test it fairly against past data: does it beat the simple 60-day rule, or is it an expensive way to get the same list? If it does beat it, they suggest a fair test: call half the high-risk customers, leave the other half alone for a month, and compare what happens. That test is how you learn whether the calls work.

Step 6: The ML engineer keeps the model running

A model in a notebook helps nobody on Monday morning. The ML engineer turns it into a service that scores every customer each night, logs every score, and raises an alarm if the model's accuracy starts to slip, for example if customer behavior changes after a price increase. They also set up a way to retrain it safely. Chapter 56 covers this work.

Step 7: The data engineer makes sure the data arrives

The model needs more than orders. It needs payments from finance, tickets from the support desk, and sales activity from the CRM, all fresh by 6 a.m. The data engineer builds pipelines that pull data from each system every night, check it (did yesterday's orders arrive? are there duplicate payments?), and load it into the warehouse. If a check fails, the pipeline stops and alerts someone *before* a wrong risk score reaches a sales rep.

Step 8: The automation or integration engineer puts it to work

A score sitting in a warehouse changes nothing. The integration engineer writes the high-risk customers into the CRM as call tasks for the right rep, making sure the same customer doesn't get a duplicate task every night. They also set up a Monday email to each rep with their own list in the body of the email, not as an attachment nobody opens. If the old finance system had no API, this is where an RPA bot might be considered, as a last resort.

Step 9: The AI engineer helps the rep prepare

Before calling Sunrise Caterers, a rep wants to know the story: last orders, open invoices, recent complaints. The AI engineer builds an assistant that drafts a short call brief from Riverstone's own data. The design matters more than the demo: the assistant may only use facts it retrieved from the company's systems, it shows where each fact came from, the rep reads and approves the brief before using it, and the team tests it regularly for invented facts. Chapters 55 and 58 teach this.

Step 10: The data architect designs how it all fits

Stand back and look at everything that now exists: pipelines, a warehouse, a shared definition, a model service, a dashboard, CRM tasks, emails, an AI assistant. The data architect decides how these pieces should fit together, so the company doesn't end up with a tangle of scripts nobody understands. Where does the risk score officially live? Who owns the definition of "active"? Should reps see only their own customers? What happens if the CRM write-back fails halfway? What does the whole thing cost each month, and is it worth it? The architect writes these decisions down, with the reasons, so the next person can understand them.

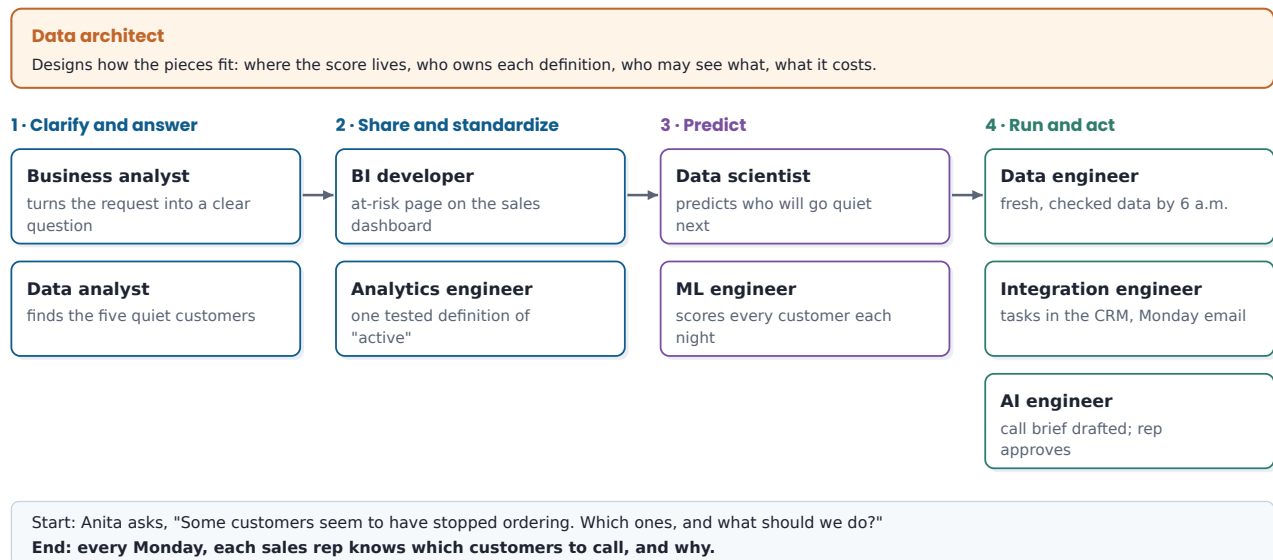


Figure 7.5 — One request, every role. Notice that the order matters: prediction is worth building only after the question is clear, the data is trusted, and the definition is agreed.

What the walk-through shows

Three things stand out.

- **Most of the value came early.** Steps 1 and 2 answered Anita's question in an afternoon. Everything after that makes the answer faster, more consistent, more predictive, or more automatic. Those are real improvements, but a company should build them only when the simpler version is in use and its limits are hurting.
- **Every role depends on the ones before it.** A model built on an unclear question, or on data nobody checks, produces confident nonsense.
- **Automation appears at almost every step.** The subscription, the tests, the nightly scoring, the pipeline, the CRM tasks, the email: each one removes a manual step someone would otherwise repeat forever.

7.7 How AI assistants are changing each role

AI assistants can now draft formulas, SQL, Python, documentation, and summaries in seconds. It's natural to wonder whether that makes these roles, or the skills in this book, less needed. The pattern so far is more useful than either hype or fear.

Adoption is high, and trust is limited. In Stack Overflow's 2025 Developer Survey, a large survey of people who write code, 84% of respondents said they use or plan to use AI tools in their work, up from 76% the year before. In the same survey, more respondents said they distrust the accuracy of AI output (46%) than trust it (33%). The survey covers developers generally, not data roles specifically, but the message fits data work well: people use these tools constantly, and they check what the tools produce.

Demand for data skills is still growing. The World Economic Forum's *Future of Jobs Report 2025*, based on a survey of employers, lists big data specialists and AI and machine learning specialists among the fastest-growing jobs to 2030, and data entry clerks among the fastest-declining. Employers in the same report rate analytical thinking as the most sought-after core skill. Read the direction, not the exact rankings: the work that is shrinking is re-typing data; the work that is growing is understanding and using it.

Here's how that plays out role by role.

Role	What AI assistants speed up	What stays your responsibility
Data analyst	First drafts of formulas and queries; explaining an unfamiliar table; summarizing results	Knowing which question matters; checking the numbers; telling the manager what to do
Business analyst	Meeting notes, first drafts of requirements and user stories	Noticing what stakeholders didn't say; resolving disagreements; deciding what's in scope
BI developer	Suggesting charts; drafting measures	Choosing what the dashboard should help people decide; making sure the numbers match the source
Analytics engineer	Drafting tests and documentation	Getting departments to agree on one definition
Data scientist	Boilerplate code; lists of candidate features	Framing the problem; spotting data leakage; designing fair tests; saying "the model isn't worth it"
ML engineer	Deployment and monitoring scripts	Deciding what to monitor, and what to do when a model degrades
Data engineer	Pipeline code; translating SQL between databases	Designing for failure; agreeing data contracts with source-system owners
Automation / integration	Building flows from a plain-language description	Handling exceptions, permissions, and what happens when a step fails silently
AI engineer	Much of the code itself	Evaluating quality and safety; deciding where a human must approve
Data architect	Drafting design documents; comparing options	Making trade-offs and being accountable for them

The right column is the durable part of each job. Notice what those tasks have in common: they need context about the business, judgment about trade-offs, and someone who is accountable when things go wrong. The left column holds tasks with a checkable right answer, and those are what assistants do well, *if* someone who understands the work checks them.

That leads to the rule this book follows throughout: **you are responsible for every number you deliver, whoever or whatever typed the formula.** An assistant that writes a query you can't read hasn't saved you time; it has moved the risk to the moment your manager acts on a wrong number. So the book teaches you to read, write, and check the work yourself, then shows how to use assistants to go faster. Chapter 6 covers learning with AI assistants without letting them think for you, and Chapter 26 covers using them well at work.

Watch out: "almost right" is the dangerous kind of wrong. An AI-drafted query that fails with an error is harmless: you notice. One that runs and returns a plausible total, while quietly counting cancelled orders, is the one that reaches the board. Hand-check one row and reconcile to a known total, every time.

7.8 Where you are right now

Wherever you're reading from (a spreadsheet job, an operations role, a college course, or a complete standing start), you're somewhere on this map. Maybe you already answer Question 1 in Excel every week without calling it analysis. Maybe you're the person who automated a painful report and became the office's unofficial automation analyst.

Locate yourself truthfully, and you'll see that the next step is usually closer than it looks. Chapter 8 gives you the tool to plan it: a career tree that shows which skills unlock which roles, what each role's day looks like, how to read a job description, and which parts of this book to read for your goal.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Choosing a target role by title or by what sounds impressive	You're learning deep learning, but the jobs you apply for want SQL and dashboards	Pick a role by the question it answers and what it produces each week (sections 7.1-7.2)
Assuming a job title means the same thing everywhere	Two "data analyst" postings ask for completely different skills	Read the responsibilities and outputs, not the title; Chapter 8 shows how
Thinking data science is "the senior version" of analysis	You feel you must leave analysis to progress	Senior analysts are highly valued; science and engineering are branches, not promotions (section 7.3)
Skipping the analyst core to reach a "higher" branch	Models and pipelines built on questions you can't frame or data you can't check	Build the trunk first: spreadsheets, SQL, business sense
Treating automation as someone else's job	The same report takes you hours every week	Every role automates; start with refreshable reports and scheduled delivery (section 7.4)
Automating a report without owning it	It fails silently when a source file changes, and nobody notices for weeks	Assign an owner, add a check, and write down what to do when it breaks
Jumping to prediction before the basics work	A churn model nobody uses, while "active customer" still has three definitions	Clarify, answer, share, and standardize before you predict (section 7.6)
Believing one team structure is always best	Endless reorganizations, same complaints	Match the structure to the company's size and pain; know each structure's typical failure (section 7.5)

Mistake	Symptom	Fix
Assuming AI assistants make core skills unnecessary	You can't tell when a drafted query is wrong	Learn to read and check the work; use assistants to go faster, not to replace understanding
Pasting AI output into a report unchecked	A plausible number that doesn't reconcile to any known total	Hand-check a row and reconcile to a total before anything leaves your hands
Picking RPA because it's quick to demo	A bot that breaks whenever a screen changes	Use an API or a direct integration where one exists; RPA only as a last resort
Waiting to "finish learning" before locating yourself	Months of study with no clear target	Place yourself on the map now; it will change, and that's fine

In the real world: Anita wants to hire a data scientist

Anita Rao came back from an industry conference with a clear idea. "Everyone is doing AI," she told Meera Iyer on Monday morning. "We need a data scientist. Someone who can predict which customers we're about to lose. Can you help me write the job advertisement?"

Meera had been at Riverstone long enough to inherit Imran's Friday file, learn SQL, and become the person everyone asked for numbers. She didn't want to say no to her boss. She also didn't think a data scientist was what Riverstone needed first.

"Before we write it," she said, "can I show you what the job would involve day to day? It'll take fifteen minutes."

She drew the four questions on the whiteboard and asked Anita which one she needed answered. Anita pointed at the second: *what will happen?* Then Meera opened the year's data and ran the query from her customer review. Five customers had gone 60 days without an order. Two of them, Sunrise Caterers and Om Sai Provisions, used to order every five or six weeks and had been silent for months. Nobody in sales had noticed.

"We didn't need a prediction to catch those two," Meera said. "We needed someone to look, every week. A model would have flagged them earlier, maybe. But right now the list isn't even on anyone's desk."

Anita frowned. "So you're saying we don't need a data scientist."

"I'm saying we don't need one yet," Meera said. She walked through what a data scientist would need to be useful: a clear definition of a lost customer, which Riverstone didn't have (sales and finance used different rules); clean history joined across orders, payments, and support complaints, which currently lived in three places, one of them an Excel file; and somebody to act on the scores every week. "Without those, a data scientist would spend their first six months doing my job and a data engineer's job, and they'd probably leave, because that's not the work they were hired for."

Then she listed the manual work she did every week: 40 minutes each morning on the Daily Sales Flash, most of a Friday on the sales pack, and copying overdue invoices from the finance system into emails for reps. "If we hire an analyst who can automate, those hours come back in the first three

months. That person sets up the at-risk list as a Monday email to every rep, fixes the definition of 'active customer' with finance, and gets the data into one place. Once that's running and we have a couple of years of clean history, a data scientist has something to work with."

Anita was quiet for a moment. "The conference speaker made it sound like step one."

"For a company with millions of customers and a data platform, it might be," Meera said. "For us, it's step five."

They rewrote the advertisement together. The title became *Data Analyst (Sales Analytics and Automation)*. The responsibilities listed the Monday at-risk list, a sales dashboard, agreed definitions with finance, and automating recurring reports. The skills section asked for SQL, Excel or Google Sheets, a BI tool, and "experience automating a recurring report, with any tool". Under *Nice to have*, Meera added "interest in forecasting and customer analytics", so the new hire could grow toward the role Anita had first imagined.

What made this work.

- **Meera answered the real need, not the requested title.** Anita wanted to stop losing customers. A title was her guess at the solution.
- **She used the map.** Framing the conversation around the four questions and the order of roles from section 7.6 turned an opinion ("I don't think we need that") into a plan.
- **She showed evidence from the data.** Two lapsed regular customers nobody noticed made the case better than any argument.
- **She didn't kill the ambition.** The data-science path stayed in the plan, in the right place.

Tools

This chapter needs no software. For the project you'll need:

- **Pen and paper**, or a free diagram tool such as **diagrams.net** (also called draw.io) or **Excalidraw**, to draw your team map and a request's journey.
- **A few job postings** from any job portal, for exercise 2 and the stretch goal. Save or print them; postings disappear.
- **Companion file (optional):** `companion/mysql/ch07_queries_mysql.sql` holds the section 7.6 query in PostgreSQL and MySQL form, for readers who already have the practice databases loaded (Chapter 12, section 12.3, explains the setup).

The project: map the data work around you

Goal: a one-page map that shows who does which data work in an organization, how a request travels, and where manual work hides. It's a small portfolio piece, and it's excellent preparation for "tell me about how your team works" in an interview.

Option A: your own workplace or college. Use a real team you know.

Privacy reminder. Don't put colleagues' names, salaries, or confidential systems in anything you share publicly. Use role names ("sales coordinator", "finance team") instead of people's names, and leave out anything your employer wouldn't want outside the building.

Option B: Riverstone Supplies. Use the people and systems you've met in Chapters 1, 2, and this chapter: Anita, Vikram, the sales executives, Meera, the Friday file, and the Daily Sales Flash.

Steps

1. **List the recurring data requests.** Write down at least five reports, lists, or numbers people regularly ask for. For each, note who asks, how often, and in what form it arrives (a file, an email, a dashboard, a phone call).
2. **Tag each request with one of the four questions.** Most will be Question 1. That's normal.
3. **Name the roles doing the work today,** whatever their titles. Where one person covers several roles, say so ("Meera: analyst, BI developer, and automation analyst").
4. **Draw the team structure.** Is it centralized, embedded, hub-and-spoke, or "one person doing everything"? Sketch it in the style of Figure 7.4.
5. **Trace one request from start to finish,** in the style of Figure 7.5: who asks, who clarifies, who pulls data, who checks it, how it's delivered, who acts on it.
6. **Mark every manual step in red:** exports, copy-paste, re-typing, emailed attachments. Estimate the minutes each one takes, and multiply by how often it happens in a year.
7. **Write three sentences:** the biggest risk in this flow, the single automation that would save the most time, and which role you'd hire (or become) next, with one reason.

Stretch goals

- Add where AI assistants are used today (or could be), and what someone checks before the output is used.
- Collect three job postings for the role you picked in step 7 and underline what each would produce in its first three months.
- Redraw the map for the organization as it might look in three years, and write down which structure you'd recommend and why.

You've got it when...

- ☐ You can name the four questions and place any data job title under one of them.
- ☐ You can describe what each of the ten roles produces, in one sentence each, without using its title.
- ☐ You can explain why SQL is useful in every role on the map.
- ☐ You can sketch the tree in Figure 7.2 from memory and explain why science and engineering are peers.

- ☐ You can explain the difference between an automation analyst, an RPA developer, and an integration engineer, and when RPA is the wrong choice.
 - ☐ You can give one example of automation inside a role that isn't an automation role.
 - ☐ You can compare centralized, embedded, and hub-and-spoke teams, including each one's typical failure.
 - ☐ You can follow a business request through the roles in a sensible order, and explain why prediction comes after a clear question and trusted data.
 - ☐ You can say, for your target role, what AI assistants speed up and what stays your responsibility.
 - ☐ You've placed yourself on the map, and can say which role you're aiming for next.
-

Recap

- **Orientation** comes before tools: know what each role does before choosing what to learn.
 - Every data job answers one of **four questions**: what happened; what will happen, and why; how data moves, gets served, and gets put to work; and how the whole system should be designed.
 - The book follows **ten roles**: data analyst, business analyst, BI developer, analytics engineer, data scientist, ML engineer, data engineer, AI engineer, automation analyst / RPA developer / integration engineer, and data architect. Judge roles by what they produce, not by title.
 - **SQL** is the one skill every role on the map uses.
 - The field grows like a **tree**: a shared trunk (foundations and the analyst core), a branch point (Part III), two peer branches (science and engineering), and a top where they rejoin (production AI and architecture).
 - The **automation and integration track** removes manual steps and connects systems, using spreadsheet automation, low-code flows, **RPA**, APIs, and **reverse ETL**. Every data role includes some automation.
 - Data teams are **centralized, embedded, or hub-and-spoke**; each trades consistency, business context, and speed differently, and each has a typical failure.
 - Following one request through every role shows the right order: clarify, answer, share, standardize, predict, then run and act, with the architect designing the whole.
 - **AI assistants** speed up tasks with a checkable answer; context, judgment, and accountability stay with you. You're responsible for every number you deliver.
-

Practice exercises

Warm-up

1. For each request, name which of the four questions it mainly answers. (a) "How many orders did we ship late last month?" (b) "How much stock of Storage Box 25L should we make for October?" (c) "Can the website's new orders reach the ERP without anyone re-typing them?" (d) "Should the

- warehouse, the CRM, and the finance system share one customer ID, and who should own it?" (e) "Did the 5% hospitality discount actually increase orders, or would they have grown anyway?"
2. Which role from section 7.2 is each person closest to, whatever their title? (a) Priya spends her week building tested SQL tables that define revenue, margin, and active customers for every dashboard in the company. (b) Arjun connects the e-commerce platform to the ERP through APIs and makes sure no order is loaded twice. (c) Sana meets department heads, maps how purchase approvals work today, and writes the requirements for a new approval system. (d) Karthik keeps a demand-forecasting model running, monitors its accuracy, and retrains it when it drifts.
 3. A company's finance and sales teams each have their own analysts, who report to their department heads. There's no central data team. Which structure is this, and what problem would you expect to hear about in meetings?

Core

4. In the section 7.6 result, Home Plus has blank `last_order_date` and `days_since_last_order` values. What do the blanks mean, and why would it be a mistake to leave Home Plus off Anita's list?
5. Check by hand that City Needs Store's `days_since_last_order` of 284 is correct. Its last order was on 22 March 2025, and "today" is 31 December 2025.
6. Anita can make two calls this afternoon. From the five customers in section 7.6, which two should she call, and why? Use the order counts given in the analyst's notes.
7. Meera spends 25 minutes each working day copying overdue invoices from the finance system into emails for sales reps. Assuming 250 working days a year, how many hours is that per year? About how many 40-hour working weeks? Which role's skills would remove this work, and name one way they might do it.
8. Riverstone has grown. It now has two analysts in sales, one in finance, and one in operations, all reporting to their department heads. In the monthly review, sales and finance report different revenue totals for the same month. Recommend a team structure, and describe the first two things the new setup should do.
9. Rewrite this vague request as a precise question a data analyst could answer, in the style of section 7.6, step 1: "Are our hotel customers happy?" List at least three clarifying questions you'd ask first.
10. For each automation, say whether an API integration or an RPA bot is the better choice, and why.
(a) Copying new website orders into the ERP, which has a documented API. (b) Downloading a monthly statement from an old supplier portal that only offers a login page and a "Download" button, with no API.

Stretch

11. Finance says "inactive" should mean *no non-cancelled order in 90 days, including customers who never ordered*. Using only the section 7.6 result, how many customers would be on finance's list, and which one drops off? Which role would fix the problem of two definitions, and how?
12. An AI assistant drafts a query that returns *four* quiet customers under the 60-day rule, not five. Without running anything, give three likely reasons for the difference and say how you'd check each one.

- l3. Trace a new request through the roles: "Our deliveries to Chennai keep arriving late. Why, and can we stop it?" Name the roles you'd involve, in order, what each produces, and which roles you'd leave out for now, with reasons.

Think about it (no query needed)

- l4. Why does a data architect need to have walked at least partway down both the science and the engineering branches? Give one concrete decision from section 7.6, step 10, that would go wrong without that experience.
- l5. Some people argue that automation shouldn't be a separate track, because every role automates. Others argue it deserves its own roles. Make the best case for each view in two or three sentences, then say which fits a company like Riverstone today.

Key terms

orientation · track · role · data analyst · business analyst · BI developer · analytics engineer · data scientist · ML engineer · data engineer · AI engineer · automation analyst · RPA developer · integration engineer · data architect · dashboard · model · pipeline · BI (business intelligence) · MLOps · LLM (large language model) · SQL · data governance analyst · data steward · data quality analyst · RPA (robotic process automation) · reverse ETL (data activation) · Daily Sales Flash · centralized team · embedded (decentralized) team · hub-and-spoke (federated) team

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 8, The Career Tree: How Skills Unlock Roles**, turns this map into tiers in the book's part order, with a skills matrix per role, a day in the life of each role, how to decode a job description, the reader pathways table, and entry routes for freshers, career switchers, and internal moves.
- **Chapter 9, How Expertise Actually Forms**, is honest about the timeline, and shows how to practice, build a portfolio, find feedback, and get through plateaus.
- **Chapters 12 and 13** teach the SQL behind section 7.6's query, and the better "customer rhythm" version of the at-risk list.
- **Chapters 19 and 20** start the automation thread in earnest: spreadsheet automation, and reports delivered by email and on schedule. **Chapter 25** covers the business analyst track, including finding automation opportunities.
- **Chapters 51, 58, and 63** take automation to the engineering, AI, and architect levels. **Chapter 66** returns to building and structuring data teams.
- **Part VIII, Chapter 68, How Data Hiring Works**, explains the interview rounds for each of these roles. Every role in this chapter has matching question banks in Chapters 70–82.

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (a) Question 1, what happened: it describes last month. (b) Question 2, what will happen: it's a forecast. (c) Question 3, how data moves and gets put to work: it's an integration between two systems. (d) Question 4, how the whole system should be designed: shared IDs and ownership across systems are architecture decisions. e. Question 2, *why*: it asks whether the discount *caused* the increase, which needs an experiment or a careful causal analysis. The common wrong answer is Question 1, because it sounds like "what happened to orders". Describing that orders rose is Question 1; proving the discount caused it is Question 2.

2. (a) Priya is an **analytics engineer**: tested, shared definitions used by every report. (b) Arjun is an **integration engineer**: API connections and duplicate-safe loading. (c) Sana is a **business analyst**: process mapping and requirements. (d) Karthik is an **ML engineer**: keeping a model running, monitored, and retrained. Titles at their companies might be anything; the outputs tell you the role.

3. It's an **embedded** (decentralized) structure. Expect the typical failure from the table in section 7.5: the departments' numbers disagree ("why does finance's revenue differ from sales?"), and similar reports get built more than once.

4. The blanks are NULLs: Home Plus has no non-cancelled orders at all, so there's no latest order date and nothing to count days from. Leaving it off would be a mistake because a customer who signed up and never ordered is the easiest kind to miss and often the easiest to win: someone showed interest and nobody followed up. That's why the business analyst's requirement in step 1 explicitly included never-ordered customers, and why the query keeps rows where the latest order date is missing.

5. From 22 March to 31 March is 9 days. April 30, May 31, June 30, July 31, August 31, September 30, October 31, November 30, December 31. Total: $9 + 30 + 31 + 30 + 31 + 31 + 30 + 31 + 30 + 31 = 284$ days.
✓ The common slip is counting 22 March itself, which gives 285; days *since* the last order start counting the day after it.

6. **Sunrise Caterers and Om Sai Provisions**. Each placed four orders at a regular rhythm (roughly every five to six weeks) and then went silent for 204 and 162 days, so something has probably gone wrong, and they were worth ₹114,072.50 together in 2025. City Needs Store and Tasty Tiffins ordered only twice each, so their silence may be their normal pattern. Home Plus is worth a call too, but as a sales lead to convert rather than a customer to win back, so it can go to a sales executive. A reasonable alternative answer puts Home Plus second, if Anita's priority is new business; what matters is giving a reason.

7. $25 \times 250 = 6,250$ minutes per year. $6,250 \div 60 \approx 104.2$ hours, and $104.2 \div 40 \approx 2.6$ working weeks. This is **automation analyst** (or integration engineer) work, and an analyst who automates could do it too. Possible approaches: a query or saved report that lists overdue invoices by rep, delivered as a scheduled email with each rep's list in the email body (Chapter 20); or a low-code flow in Power Automate or n8n that runs every morning (Chapter 20); or, at a later stage, overdue flags pushed straight into the CRM as tasks (Chapter 51).

8. A **hub-and-spoke** structure fits: the analysts stay close to their departments, but a small hub (even one person to begin with) owns shared definitions and the platform. First two things: (1) agree one written definition of revenue with sales and finance (for example, net of discount, excluding cancelled orders, by order date or by invoice date, but one of them), and build it once as a shared table; (2) point both departments' reports at that shared table and reconcile the old numbers so everyone understands why they differed. Moving everyone into a fully centralized team would also fix consistency, but it would lose the business context the embedded analysts already have.
9. Clarifying questions could include: *What does "happy" mean to you: ordering more, complaining less, paying on time, or a survey score? Which customers count as "hotel": the Hospitality segment, which also includes caterers? Over what period? Compared with what: last year, or other segments? Who will act on the answer, and what decision is it for?* One precise version: **"For Hospitality customers, compare the number of orders, revenue, and support complaints per customer in the last six months of 2025 with the six months before, and list customers whose orders fell by more than half."** Any precise version is acceptable if it names the group, the measures, the period, and the comparison.
10. (a) **API integration.** A documented API is designed for systems to exchange data; it's faster, and it doesn't break when a screen's layout changes. (b) **RPA bot** is reasonable here, because there's no other way in. Treat it as fragile: monitor it, and ask the supplier whether a data feed or an emailed statement is available, so the bot can be retired.
11. Finance's rule keeps customers with more than 90 days since their last order, plus never-ordered customers: Home Plus (never), City Needs Store (284), Sunrise Caterers (204), and Om Sai Provisions (162). That's **4 customers. Tasty Tiffins drops off**, at 66 days. An **analytics engineer** would fix the problem of two definitions: agree one definition (or two clearly named ones, such as "quiet 60" for sales follow-up and "inactive 90" for finance), build it once as a tested, documented table, and point every report at it. The wrong fix is to let each team keep its own hidden rule and argue in meetings.
12. Likely reasons: (1) **Never-ordered customers were dropped.** An inner join, or a filter written as a plain `WHERE` condition on orders, removes Home Plus. Check: does Home Plus appear? (2) **Cancelled orders were counted as activity.** A customer whose only recent order was cancelled would look active. Check: look for cancelled orders after 1 November for the missing customer. (3) **A different cut-off date or comparison.** For example `<=` instead of `<`, a cut-off of 60 days from a different "today", or `CURRENT_DATE` instead of 31 December 2025. Check: compare the dates in the query with the requirement. In every case, reconcile: list the customers in both results and investigate the one that differs. Here, reason (1) is the most likely, because Home Plus is exactly the kind of row that inner joins silently lose.
13. A sensible order: **Business analyst** first, to define "late" (against the promised date? by how many days?) and find who needs the answer. **Data analyst** next, to measure how often Chennai deliveries are late, compared with other cities, and by delivery partner, route, product, or day of the week, and to suggest the likely causes. **BI developer**, if it needs watching every week, to add an on-time delivery page to the operations dashboard. **Automation or integration engineer**, if delivery status has to be copied from the delivery partners' systems by hand, to connect those systems. Leave out the **data scientist** and **ML engineer** for now: prediction is premature until the cause is understood and the

data is reliable; they might come in later to predict late deliveries before dispatch. The **data architect** isn't needed for one analysis, though they'd care if delivery data turned out to be missing from the platform. Other reasonable orders are fine if each step depends on the one before.

14. Because the architect's decisions sit where the branches meet, and each branch fails in different ways. Example from step 10: *"What happens if the CRM write-back fails halfway?"* Without engineering experience, an architect might not know that retries can create duplicate tasks unless each write can be safely repeated. Another example: *"Where does the risk score officially live?"* Without science experience, they might not realize the score needs to be stored with the model version and date that produced it, or nobody can later explain why a customer was flagged. An architect who has never met these problems tends to design systems that look clean on paper and break in production.

15. *For a separate track:* automation at scale needs specialist knowledge of APIs, integration platforms, RPA, error handling, and governance, and companies with many systems benefit from people who do it full time and own the automations. *Against:* the best automations come from people who understand the work being automated, so every role should automate its own repeated tasks, and a separate team can become a bottleneck. For a company like Riverstone today, most automation should come from analysts who automate their own reports (as in the story), with a specialist integration role added when many systems need connecting. Any answer that weighs both sides and ties the choice to the company's size is acceptable.